Коллокация, Метод Стрельбы

Современные методы управления роботами

Сергей Савин

2026

Содержание

- Напоминание: задача оптимизации траектории
- Параметризация функций: последовательность точек и сплайны
- Кубические сплайны Эрмита
- Функция стоимости и ограничения
- Коллокация: кодирование динамики в дискретных точках
- Прямое прицеливание и интегрирование Рунге–Кутты
- Пример: раскачивание перевёрнутого маятника
- Литература

Напоминание: Оптимизация траектории

Оптимизация траектории — это задача поиска *допустимой траектории*, минимизирующей выбранную функцию стоимости.

Например, мы можем сформулировать задачу как

$$\begin{aligned} \min_{x(t),u(t),T} \quad & \Phi\big(x(T)\big) + \int_0^T L\big(x(t),u(t)\big)\,dt \\ \text{s.t.} \quad & \dot{x}(t) = f\big(x(t),u(t)\big), \\ & x(0) = x_A, \qquad x(T) = x_B. \end{aligned}$$

- $\Phi(x(T))$ — это **терминальная стоимость**, которая штрафует конечное состояние.
- $L(x(t),u(t))$ — это **стоимость движения**, которая штрафует состояние и управление вдоль траектории.

Конкретный выбор Φ и L зависит от того, чего мы хотим достичь с помощью траектории.

Сплайны

Сплайн — это гладкая кусочно-полиномиальная функция.

Для заданного набора узлов

$$t_1 < t_2 < \cdots < t_N,$$

сплайн определяется на каждом интервале $[t_i, t_{i+1}]$ как

$$S_i(t) = a_i + b_i(t - t_i) + c_i(t - t_i)^2 + \cdots + d_i(t - t_i)^K.$$

Полиномы гладко соединяются в узлах. Например,

$$S_i(t_i) = S_{i+1}(t_i),$$

$$S_i'(t_i) = S_{i+1}'(t_i), \qquad S_i''(t_i) = S_{i+1}''(t_i), \quad \ldots$$

Таким образом, сплайн может представлять гладкую функцию, используя лишь конечное число параметров.

Параметризация функций: последовательность точек

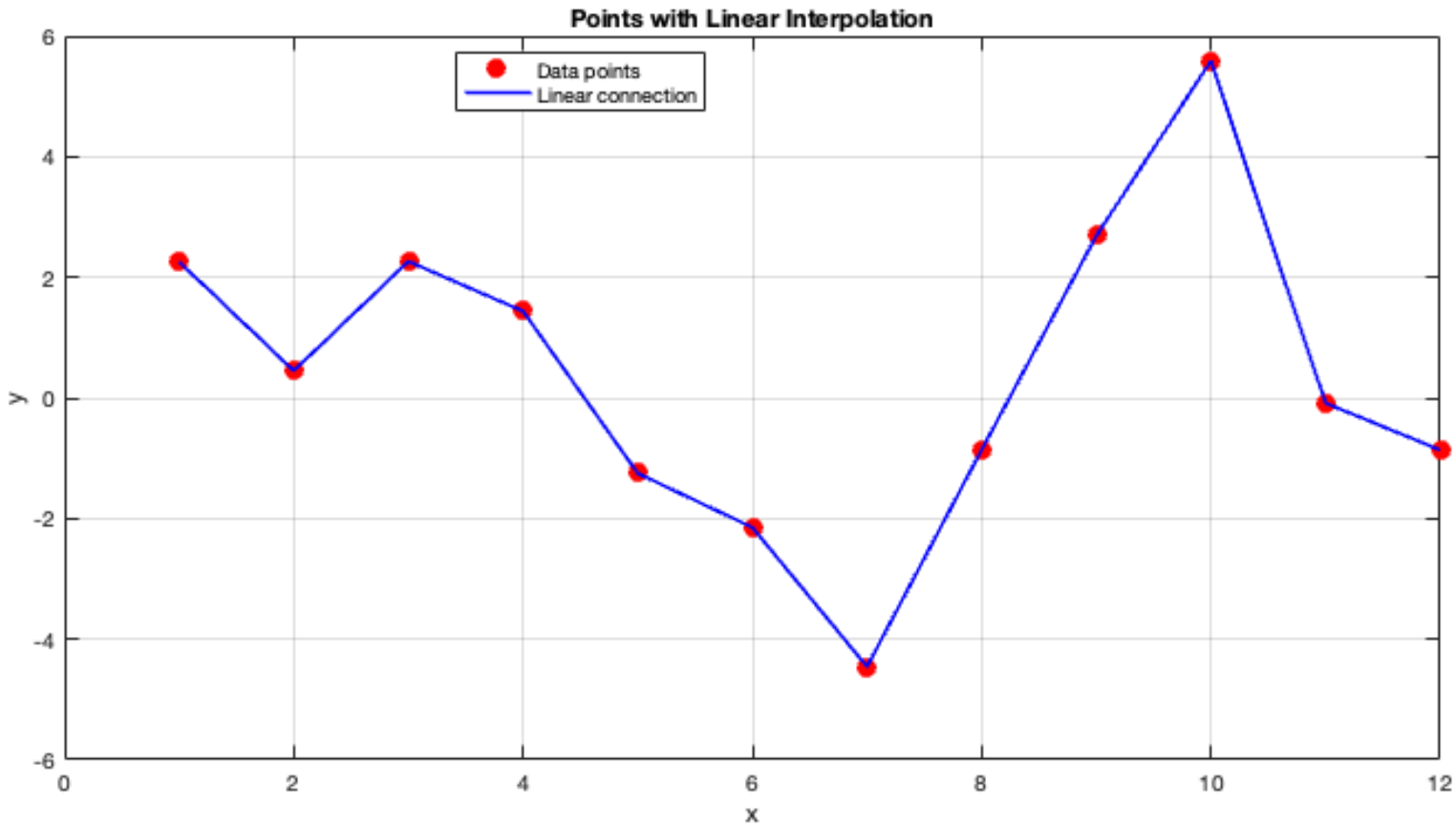


Рис. 1: Последовательность точек

Параметризация функций: сплайн

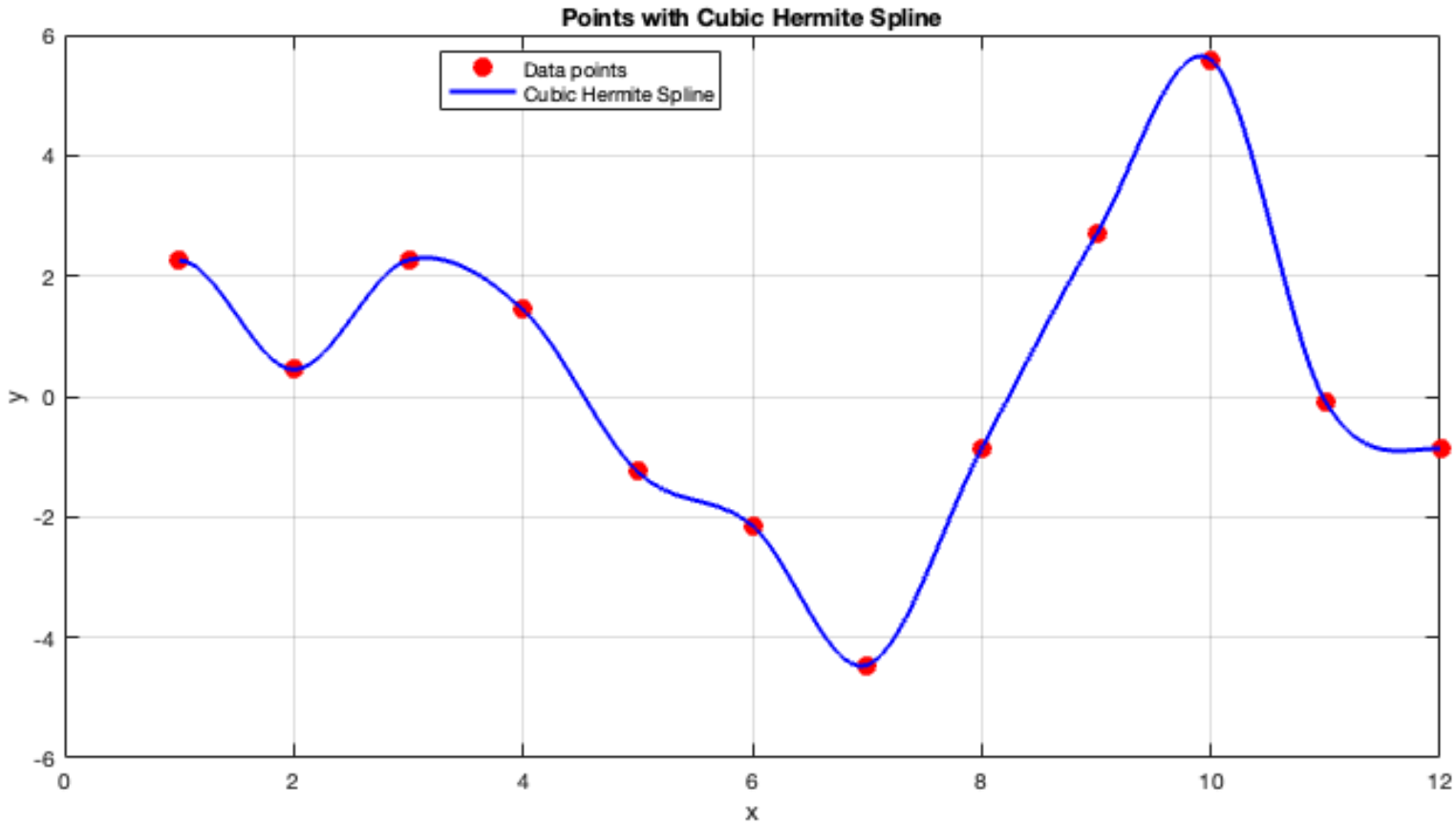


Рис. 2: Кубический сплайн Эрмита

Кубический сплайн Эрмита

Кубический сплайн Эрмита строится из полиномов третьей степени между последовательными узлами.

Для $t \in [t_i, t_{i+1}]$ определим нормированное время

$$\tau = \frac{t - t_i}{t_{i+1} - t_i}, \qquad \tau \in [0, 1].$$

Сплайн имеет вид

$$S_i(t) = P_i h_{00}(\tau) + T_i h_{10}(\tau) + P_{i+1} h_{01}(\tau) + T_{i+1} h_{11}(\tau),$$

где P_i — значение, а $(t_{i+1} - t_i)T_i$ — производная в i-м узле. Базисные функции кубического сплайна Эрмита:

$$h_{00}(\tau) = 2\tau^3 - 3\tau^2 + 1, \quad h_{01}(\tau) = -2\tau^3 + 3\tau^2,$$
$$h_{10}(\tau) = \tau^3 - 2\tau^2 + \tau, \quad h_{11}(\tau) = \tau^3 - \tau^2.$$

Эти базисные функции обеспечивают заданные значения и производные на обоих концах.

Сплайны в планировании траектории

В планировании траектории нам нужно заменить непрерывные функции $u(t)$ и $x(t)$ конечным набором параметров.

Один из вариантов — представить функции их значениями в конечном наборе точек:

$$u(t) \quad \longrightarrow \quad \{u_1, u_2, \ldots, u_N\}.$$

В качестве альтернативы мы можем параметризовать их с помощью сплайнов:

$$u(t) = u(c, t),$$

где c содержит коэффициенты сплайна или значения в узлах.

Сплайны обеспечивают гладкую аппроксимацию траектории и позволяют избежать разрывов.

Однако гладкость не всегда желательна:

“нажать на газ” $\longrightarrow$ “нажать на тормоз”

может естественно потребовать разрывного управляющего воздействия.

Функция стоимости

Функция стоимости зависит от задачи, которую мы пытаемся решить. Однако несколько общих целей часто встречаются в планировании траектории.

Для задачи перехода из x_A в x_B за N шагов мы можем хотеть:
- достичь желаемого конечного состояния: $\|x_N - x_B\|^2$;
- поддерживать близость последовательных состояний: $\sum_{i=0}^{N-1} \|x_{i+1} - x_i\|^2$;
- использовать малые управляющие воздействия: $\sum_{i=0}^{N-1} \|u_i\|^2$.

Эти цели можно объединить в единую стоимость:

$$\min \quad w_1\|x_N - x_B\|^2 + w_2 \sum_{i=0}^{N-1} \|x_{i+1} - x_i\|^2 + w_3 \sum_{i=0}^{N-1} \|u_i\|^2.$$

Веса w_1, w_2, w_3 определяют относительную важность различных целей.

Квадратичная стоимость общего вида

Распространённый выбор функции стоимости — квадратичная стоимость:

$$\min \quad x_N^\top Q_N x_N + \sum_{i=0}^{N-1} \left(x_i^\top Q x_i + u_i^\top R u_i \right),$$

где $Q_N \succeq 0, \quad Q \succeq 0, \quad R \succ 0$ — весовые матрицы.

Если $u(t)$ или $x(t)$ параметризованы с помощью сплайна, мы можем вычислить сплайн в любой желаемой момент времени t_i. Точки вычисления не обязаны совпадать с узлами сплайна.

Например, если $u(t) = a + bt$, то вычисление $J = \sum u_i^2$ в

$$t_i \in \{0, 0.5, 1\}$$

даёт

$$J = a^2 + (a + 0.5b)^2 + (a + b)^2.$$

Ограничения

Задачи планирования траектории часто включают ограничения, которые описывают, какие траектории физически или геометрически допустимы.

Распространённые примеры включают:

- Ограничения по моменту:

$$|u_i| \leq u_{\max}.$$

- Ограничения по шарнирам и скорости:

$$x_{\min} \leq x_i \leq x_{\max}.$$

- Ограничения самопересечения.
- Избегание препятствий и столкновений.
- Ограничения контакта и захвата.

Мы также обычно накладываем начальные и конечные условия как ограничения:

$$x_0 = x_A, \qquad x_N = x_B.$$

Коллокация

Перед нами остаётся задача кодирования динамики системы:

$$\dot{x} = f(x, u).$$

Коллокация обеспечивает выполнение динамики в конечном числе моментов времени, называемых *точками коллокации*.

Рассмотрим простой случай, когда $x(t)$ и $u(t)$ представлены своими значениями

$$x_0, x_1, \ldots, x_N, \qquad u_0, u_1, \ldots, u_{N-1}.$$

Аппроксимируя производную с помощью конечной разности,

$$\dot{x}(t_i) \approx \frac{x_{i+1} - x_i}{t_{i+1} - t_i},$$

мы получаем ограничения коллокации

$$\frac{x_{i+1} - x_i}{t_{i+1} - t_i} = f(x_i, u_i), \qquad i = 0, \ldots, N-1.$$

Коллокация: свойства

Подход коллокации к обработке динамики имеет отличительные свойства:

- Динамика обеспечивается только в конечном числе точек коллокации.
- Переменные решения включают состояния

$$x_0, x_1, \ldots, x_N$$

и управления

$$u_0, u_1, \ldots, u_{N-1}.$$

- В качестве альтернативы $x(t)$ и $u(t)$ могут быть представлены с помощью сплайнов или других параметризаций.
- Получающаяся задача оптимизации содержит явные ограничения, представляющие динамику системы.

Прямое прицеливание

В отличие от коллокации, *прямое прицеливание* кодирует динамику через схему численного интегрирования.

Для заданного начального состояния x_0 и управляющих воздействий $u_0, u_1, \ldots$ состояния генерируются путём распространения динамики вперёд по времени.

Например, используя метод Эйлера,

$$x_{i+1} = x_i + f(x_i, u_i)(t_{i+1} - t_i).$$

Начиная с x_0, это даёт

$$x_1 = x_1(u_0), \qquad x_2 = x_2(u_0, u_1), \qquad \ldots$$

Таким образом, состояния являются *выражениями параметров управления*, а не независимыми переменными решения.

При интегрировании методом Эйлера это алгебраически эквивалентно предыдущим дискретизированным ограничениям динамики. Различие становится важным при использовании более сложных схем интегрирования.

Интегрирование Рунге–Кутты

Вместо метода Эйлера мы можем использовать схему интегрирования более высокого порядка, такую как метод Рунге–Кутты четвёртого порядка (RK4).

Для шага по времени $h = t_{i+1} - t_i$ определим

$$
\begin{aligned}
k_1 &= f(x_i, u_i), \\
k_2 &= f\left(x_i + \frac{h}{2} k_1, u_i\right), \\
k_3 &= f\left(x_i + \frac{h}{2} k_2, u_i\right), \\
k_4 &= f\left(x_i + h k_3, u_i\right).
\end{aligned}
$$

Затем состояние распространяется согласно

$$
x_{i+1} = x_i + \frac{h}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right).
$$

Таким образом, при прямом прицеливании вся траектория получается путём повторного применения этого шага.

Перевёрнутый маятник: Коллокация

Рассмотрим маятник с приводом и динамикой

$$\dot{\omega} = -\sin(\phi) + u, \qquad \dot{\phi} = \omega.$$

Мы хотим перевести маятник из нижнего положения в верхнее: $\phi_0 = 0, \quad \phi_N = \pi, \quad \omega_0 = \omega_N = 0.$
Формулировка коллокации:

$$\min_{\phi_i, \omega_i, u_i} \quad \sum_{i=0}^{N-1} \left[w_1(\omega_{i+1} - \omega_i)^2 + w_2(\phi_{i+1} - \phi_i)^2 + w_3 u_i^2 \right]$$

$$\text{s.t.} \quad \frac{\omega_{i+1} - \omega_i}{h} = -\sin(\phi_i) + u_i,$$

$$\frac{\phi_{i+1} - \phi_i}{h} = \omega_i, \qquad i = 0, \ldots, N-1,$$

$$\phi_0 = 0, \quad \omega_0 = 0, \quad \phi_N = \pi, \quad \omega_N = 0,$$

$$|u_i| \le u_{\max}, \qquad i = 0, \ldots, N-1.$$

Здесь ϕ_i, ω_i и u_i — все переменные оптимизации.

Перевёрнутый маятник: Прямое прицеливание

Можем решить ту же задачу с помощью прямого прицеливания. Переменными решения теперь являются только управляющие воздействия: $u_0, u_1, \ldots, u_{N-1}$. Начиная с $\phi_0 = 0, \quad \omega_0 = 0$, мы распространяем динамику с помощью RK4. Получающаяся задача оптимизации:

$$\min_{u_0,\ldots,u_{N-1}} \quad \sum_{i=0}^{N-1} \left[w_1(\omega_{i+1} - \omega_i)^2 + w_2(\phi_{i+1} - \phi_i)^2 + w_3 u_i^2 \right] \tag{1}$$

$$\text{s.t.} \quad \begin{bmatrix} \omega_{i+1} \\ \phi_{i+1} \end{bmatrix} = \text{RK4}\left(\begin{bmatrix} \omega_i \\ \phi_i \end{bmatrix}, u_i, h \right), \tag{2}$$

$$\phi_N = \pi, \qquad \omega_N = 0, \tag{3}$$

$$|u_i| \leq u_{\max}, \qquad i = 0, \ldots, N-1. \tag{4}$$

Здесь ϕ_i и ω_i генерируются динамикой и не являются независимыми переменными оптимизации.

Литература

- Russ Tedrake – MIT: Underactuated Robotics: Trajectory Optimization, ch. 10.

Слайды доступны на Github:

github.com/SergeiSa/Advanced-Control-2026

